Variables

All programs, no matter how humble, need to manipulate data at some point in their lifecycle. Even the most universally simple program contains data. Here's a PHP version of the venerable "hello world."

```
echo "Hello World";
```

```
Output:
Hello World
```

The text between the double-quote characters is the data, a string literal, and the `echo` keyword is an instruction to output that data. Exactly where this data goes depends on how PHP is executing on your system. In a Unix/Linux shell context (where PHP is running in the `cli` or `cli-server` SAPI), the data will output to STDOUT (which is normally displayed in your console window).

"Hello world" is a nice way to confirm PHP is installed correctly, but not very useful beyond that. Real Programs™ will need to store, retrieve, and perform operations on their data. In the example above, after `echo` has done its work, the data is gone. If we needed to reuse that string of characters, we'd need to type it out again. What if you wanted to keep hold of it for future use or combine it with other data?

To persist data within your program, you'll need to create variables. These are used to store and retrieve data. In PHP, the `$` character prepended to a string forms the *symbolic name* used to read and write data to a memory address (don't worry, PHP manages memory assignment for you; no need to worry about things like stack and heap here).

```
$txt = "Hello World";

echo $txt;
```

```
Output:
Hello World
```

Let's break down what's happening. We define a *string literal* – `"Hello world"` – and assign this data to the variable `$txt`. Under the hood, PHP will deal with allocating memory and storing the data, determining its type, and keeping track of it via the `$txt` variable name. We then use this variable with the language construct `echo`. PHP retrieves the data from memory and uses it as per the normal behavior of `echo`, which sends the string of text for display somewhere.